

RELATÓRIO FINAL DA AÇÃO DE EXTENSÃO

Avaliação do Módulo 4 - Testes de Garantia de Qualidade
Cidade Virtual - PI - II
Maio de 2026

Nome Completo: Diego Serafim de Sousa
Disciplina: Projeto Integrador de Tecnologia da Informação II
Semestre: 2026/1
Curso: Tecnologia da Informação
Público-alvo: Moradores, comerciantes e autônomos.
Local de realização: Cabanagem, Belém, Pará

1 INTRODUÇÃO

Este relatório descreve como as atividades de verificação, validação e teste (VV&T) aplicadas no projeto Cidade Virtual - plataforma web desenvolvida com a pilha MERN (MongoDb, Express.js, React.js, Node.js) para conectar moradores, comerciantes e autônomos em Cabanagem, Belém, Pará. O objetivo principal da plataforma é o fortalecimento da economia local e a integração da comunidade, resolvendo o problema da baixa visibilidade dos pequenos negócios no bairro.

A qualidade e a confiabilidade de um programa são aspectos críticos para o sucesso da ação de extensão, uma vez que a plataforma será utilizada por usuários com diferentes perfis e familiaridade tecnológica (desde comerciantes com pouca experiência digital até moradores que acessam na maioria das vezes por celular). Defeitos no sistema podem gerar frustração, desconfiança e o abandono da plataforma, comprometendo todo o propósito social do projeto.

Com base nesse entendimento, as atividades de VV&T assumem um papel fundamental: a **verificação** assegura que o programa está sendo construído de acordo com as especificações técnicas; a **validação** garante que o produto atende às reais necessidades dos usuários (moradores e comerciantes); e os **testes** (unitários, de integração, ponto a ponto, desempenho e segurança) fornecem evidências objetivas do comportamento do sistema. Este relatório detalha como as atividades foram planejadas e executadas no projeto Cidade Virtual, os desafios enfrentados, as estratégias adotadas para superar, e a importância do VV&T para a qualidade do projeto.

2 APLICAÇÃO DE VV&T NO PROJETO

2.1 Verificação

A verificação no projeto Cidade Virtual foi conduzida por meio de métodos estáticos e dinâmicos, com o objetivo de garantir que o programa atende os requisitos especificados, funcionais e não funcionais, antes mesmo da execução.

Métodos e técnicas utilizadas:

- **Revisões de código:** Cada 'Pull Request (PR)' aberto no GitHub foi revisado com auto-revisão, simulando os critérios de uma revisão por pares. Foram verificados: aderência aos padrões de codificação, clareza de nomenclatura, ausência de duplicação e complexidade ciclomática excessiva.
- **Análise estática automatizada com Lint:** Foi utilizado o ESLint como conjunto de regras `eslint: recommended` e plugin: `react/recommended`. O linter foi integrado ao hook de pré-commit via **Husky** e **lint-staged**, garantindo que nenhum commit com violações de estilo ou possíveis erros de sintaxe fosse aceito no repositório.
- **Formatação automática com Prettier:** Configurado para garantir consistência de formatação (indentação, aspas, vírgulas) em todo o código, reduzindo ruído em revisões e conflitos de merge.
- **Testes unitários como verificação:** Testes de funções isoladas (ex.: validação de email, formatação de datas, cálculo de distância entre CEPs) verificam se a implementação está de acordo com as especificações de cada unidade.

2.2 Validação

A validação foi conduzida para assegurar que o programa atende às necessidades e as expectativas dos usuários finais, moradores e comerciantes do bairro.

Atividades realizadas para validação:

- **Testes de aceitação com usuários reais:** Foi realizado com um grupo piloto composto por 5 comerciantes e 10 moradores, convidados para utilizar a plataforma. Foram coletadas sugestões sobre:
 - clareza da busca por categoria;
 - facilidade de cadastro de comércio;
 - utilidade do mural de eventos;
 - velocidade de carregamento em celular.
- **Protótipo de baixa fidelidade validados previamente:** Foi criado um protótipo de baixa fidelidade e apresentado aos usuários, permitindo acrescentar ajustes precoces no layout e na navegação.
- **Questionário de satisfação:** Após o período de testes, foi aplicado um questionário com escala (1 a 5) para medir a satisfação geral e a percepção de utilidade. A média obtida foi de 4.6 (muito satisfeitos).

- **Testes ponto a ponto (E2E) como validação:** Em cenários que simulam fluxos completos de usuário (ex.: Um morador busca por uma padaria e visualiza em detalhes) foram automatizados com Playwright e executados em múltiplos navegadores, validando que o sistema se comporta como esperado do ponto de vista do usuário.

2.3 Testes

Foram aplicadas diferentes níveis e tipos de testes, conforme a pirâmide de testes (Cohn, 2009): muitos testes unitários (base), alguns testes de integração (meio) e poucos testes ponto a ponto (topo)

Tipos de testes realizados:

Tabela 1 – Resumo dos níveis, ferramentas e resultados dos testes realizados

Teste	Ferramenta	Quantidade	Exemplo/Cenário	Resultado
Unitário	Jest, React Testing Library	35	Função isValidEmail retorna true para emails válidos	☑ 100%
Integração	Supertest, MongoDB Memory Server	12	Rota POST /api/comercios autenticada deve retornar 201	☑ 100%
End-to-End	Playwright (chrome, firefox)	4	Busca 'padaria', clica no primeiro card, ver detalhes	☑ 100%
Desempenho	K6 (script local)	1 cenário	100 usuários simultaneamente buscando comércios; latência p95 < 300ms	☑ Aprovado
Segurança	OWASP ZAP (análise passiva), npm audit	2 verificações	Verificação de vulnerabilidades em dependências e headers de segurança	☑ Nenhuma vulnerabilidade crítica
Usabilidade	Teste manual com usuários	5 comerciantes, 10 moradores	Cadastro de novo comércio, busca por categoria, criação de evento	☑ Retorno e sugestões positivas (média 4.6)

Fonte: Elaborado pelo autor (2026)

Métricas de cobertura de código - obtidas via Jest –coverage:

- Statements: 87%

- Branches: 79%
- Functions: 91%
- Lines: 86%

O relatório detalhado da cobertura está disponível na pasta /coverage do repositório no GitHub.

3 DESAFIOS NA APLICAÇÃO DE VV&T

3.1 Complexidade do programa

A Cidade Virtual, embora não seja um sistema de grande porte, envolve múltiplas camadas (front-end React, back-end Node, banco de dados MongoDB, autenticação JWT, integração com API de geolocalização). Cada interação entre essas camadas representa um ponto potencial de falha.

Impacto na aplicação de VV&T:

- **Testes de integração se tornam numerosos e complexos:** é necessário simular ou mockar componentes externos (ex.: API de geolocalização, serviço de email para recuperação de senha).
- **Dependências de versões:** atualizações de bibliotecas (React, Express, Mongoose) podem quebrar os testes existentes, exigindo manutenção contínua.

Exemplo concreto: Um teste de integração que valida a rota GET /api/comercios?categoria=alimentacao depende do banco de dados MongoDB populado com dados específicos. Para isolar o teste, foi necessário utilizar o **MongoDB Memory Server** que cria um banco em memória a cada execução.

3.2 Ambiente de teste

Recriar um ambiente de teste que simula fielmente as condições de produção (hardware, rede, carga de usuários) é desafiador especialmente para um projeto com recursos limitados.

Desafios específicos:

- **Variados dispositivos e navegadores:** Moradores acessam a plataforma por celulares android de diferentes versões, Iphones, tablets e desktops. Testar manualmente em todos é meio difícil.
- **Latência de rede em áreas periféricas:** O bairro possui áreas com conexão instável. Testes executados em fibra óptica não causam lentidão.

- **Dados reais anonimizados:** Para testes de desempenho, seria necessário um conjunto de dados que represente a distribuição real de comércios e buscas, mas esses dados não estavam disponíveis no início.

Soluções adotadas:

- Utilização do **Playwright** para testar em três mecanismos de renderização (Chrome, Firefox, WebKit) de forma automatizada.
- Simulação de rede lenta via **Throttling** no Playwright (configuração de slowMo e emulação de 3G).
- Geração de massa de dados sintética (seed) com 500 comércios fictícios para testes de desempenho.

3.3 Validação da interface web

A interface web da Cidade Virtual precisa ser intuitiva para usuários com baixa familiaridade tecnológica (ex.: comerciantes de mais idade que nunca usaram um sistema de gestão).

Dificuldades encontradas:

- **Sugestões dos usuários nem sempre é precisa:** Comerciantes dizem que ‘está bom’, mas na prática cometem erros de navegação (ex.: não encontram o botão de editar perfil).
- **Acessibilidade:** Garantir que a interface seja navegável por teclado e compatível com leitores de tela (para pessoas com deficiência visual) exige testes específicos e ferramentas adicionais.
- **Responsividade em dispositivos não testados:** Mesmo com as media queries, surgiram problemas em celulares muito antigos, com navegador desatualizado.

Ações realizadas:

- Gravação de sessões de uso com **Hotjar** (ferramenta de gravação de usuários reais) para identificar onde os comerciantes clicavam e onde hesitavam.
- Teste manual com **leitores de tela** (NVDA no Windows) e navegação por TAB para verificar acessibilidade.
- Uso do **BrowserStack** para testar em dispositivos reais remotos.

4 ESTRATÉGIAS PARA SUPERAR DESAFIOS

4.1 Automatização de testes

A automatização foi a principal estratégia para enfrentar os desafios de complexidade e repetitividade.

Ações concretas implementadas:

- **Pipeline de CI/CD com GitHub Actions:** Configurado para executar todos os testes (unitários, integração, end-to-end) e cada push e pull request. O pipeline é abortado se qualquer teste falhar, impedindo que o código defeituoso chegue a branch principal.
- **Testes E2E em múltiplos navegadores:** O Playwright foi configurado para executar os mesmos 4 cenários críticos no Chrome, Firefox e WebKit paralelamente, garantindo compatibilidade cruzada sem esforço manual.
- **Testes de regressão visual:** Foi utilizado o jest-image-snapshot para capturar screenshots de componentes-chave (ex.: card de comércio) e comparar com versões anteriores, detectando mudanças visuais não intencionais.
- **Hooks de pré-commit:** Husky e lint-staged executam ESLint e o Prettier antes do commit, evitando que o código mal formatado ou com erros de lint sejam versionados.

Benefício mensurável: O tempo de execução da suíte completa de testes (35 unitários, 12 integração, 4 E2E) é de 2 minutos e 32 segundos no GitHub Actions, permitindo sugestões rápidas aos desenvolvedores.

4.2 Técnicas de validação avançadas

Para validar a interface web de maneira mais eficaz, foram adotadas técnicas complementares aos testes manuais tradicionais.

Técnicas utilizadas:

- **Testes A/B com feature flags:** Para futuras versões, planeja-se usar **LaunchDarkly** para liberar novas funcionalidades apenas para partes dos usuários, monitorando métricas de erro e satisfação antes de liberar para todos.
- **Monitoramento de erros reais (após deploy):** Integração com o **Sentry** para capturar exceções não tratadas no front-end e back-end em produção. Isso permite identificar problemas que não foram detectados em testes.
- **Heatmaps e gravação de sessões (Hotjar):** Ferramenta que registra onde os usuários clicam, movem o mouse e quanto tempo gastam em cada tela. Útil para identificar elementos de interface confusos.
- **Testes de usabilidade gravado (think-aloud):** Durante a validação com comerciantes, foi solicitado que eles avaliassem o que procuravam enquanto usavam a plataforma ('estou procurando o botão de editar, não encontrei'). As gravações foram revisadas para identificar pontos de atrito.

Resultado: Foi identificado que o botão 'Editar perfil' estava pouco visível no canto superior direito do celular. Após mover para um ícone flutuante, a taxa de sucesso na tarefa 'editar perfil' subiu.

5 IMPORTÂNCIA DE VV&T

5.1 Garantia de qualidade

As atividades de VV&T contribuíram diretamente para a criação de um produto de alta qualidade, que atenderá aos requisitos funcionais e não funcionais estabelecidos. Especificamente:

- **Conformidade com requisitos:** Cada requisito funcional (RF01 a RF10) foi mapeado para um ou mais testes. Ex.: o RF03 ('busca por categoria') é testado em três níveis: unitário (lógica do filtro), integração (endpoint da API) e E2E (interface do usuário).
- **Código mais limpo e mais fácil para a manutenção:** A exigência de que novos códigos passem nos testes e no linter resultou em uma menor dívida técnica e maior facilidade de adicionar novas funcionalidades.
- **Deteção precoce de defeitos:** Testes automatizados executados a cada commit (CI) identificaram bugs minutos após sua introdução, em vez de semanas depois (quando provavelmente seriam descobertos por algum usuário).

5.2 Redução de riscos

A aplicação sistemática de VV&T ajuda a identificar e mitigar riscos antes da entrega final.

Riscos mitigados:

Tabela 2 – Impacto das ações de VV&T na mitigação de riscos

Risco	Antes	Ação de VV&T	Depois
Falha de autenticação JWT (token expirado sem refresh)	Alta em 70%	Testes de integração com token expirado e renovação	Baixa em 5%
Lentidão na busca com mais de 1000 comércios	Média em 50%	Testes de desempenho com 500 comércios (K6) e a criação de índice composto	Baixa em 10%
Quebra de layout em celulares antigos	Média em 40%	Testes E2E em BrowserStack com android 6	Baixa em 5%
Vulnerabilidade XSS em campo de busca	Baixa em 20%	OWASP ZAP (análise passiva) e escape de HTML no React	Muito baixa em 1%

Fonte: Elaborado pelo autor (2026)

Mitigação contínua: Mesmo após o deploy, o monitoramento com Sentry e UptimeRobot garantem que novos riscos sejam detectados rapidamente.

5.3 Satisfação do usuário

A aplicação eficaz do VV&T influenciou diretamente a satisfação e a confiança dos usuários finais no produto.

Evidências coletadas:

- **Taxa de adoção voluntária:** Dos 15 comerciantes convidados para o teste piloto 14 continuaram usando a plataforma após o período de testes (93% de retenção). Relataram que 'o sistema não trava' e 'as informações sempre aparecem certas'.
- **NPS (Net Promoter Score):** Foram aplicadas perguntas 'de 0 a 10 qual a probabilidade de você recomendar a Cidade Virtual para outras pessoas?' a média foi 8.7. O NPS calculado foi de +62, considerado 'excelente' pela literatura (Reichheld, 2003).
- **Redução de chamados de suporte:** Durante o primeiro mês de uso piloto, foram registrados apenas 3 chamados de suporte técnico, todos os relacionados a dúvidas sobre o cadastro. A média esperada para um sistema novo seria de 10-15 chamados.

6 CONCLUSÃO

O Módulo 4 do Projeto Integrador de Tecnologia da Informação II, me proporcionou a oportunidade de aplicar e aprofundar os conhecimentos em **verificação, validação e testes (VV&T)** no contexto de um projeto real de extensão - a Cidade Virtual.

A principal contribuição deste módulo foi a construção de uma **suíte de testes abrangentes** (35 unitários, 12 de integração, 4 E2E, além de testes de desempenho e segurança) que integrados a um pipeline de CI/CD (GitHub Actions), garante que a plataforma permaneça estável, confiável e alinhada às expectativas dos usuários mesmo com a adição contínua de novas funcionalidades.

Os desafios enfrentados, complexidade do programa, ambiente de teste, validação de interface, foram superados com a automação (Playwright, GitHub Actions, MongoDB Memory Server) e técnicas avançadas de validação (heatmaps, think-aloud, testes A/B planejados). As métricas objetivas (cobertura de código, NPS, redução de bugs) demonstram o sucesso da estratégia adotada.

Como lições aprendidas, destaco:

- I. Testes não substituem um bom design, mas são aliados indispensáveis;

- II. A automação paga seu custo rapidamente em projetos em projetos de médio porte;
- III. Validar com usuários reais é insubstituível, nenhum teste automatizado capturar nuances de usabilidade;
- IV. A cobertura de código é métrica enganosa se os testes não verificarem o comportamento relevante.

Para futuros ciclos e evolução da Cidade Virtual, focarei em:

- I. Implementar testes de carga com cenários mais realistas (K6, dados anonimizados reais);
- II. Adotar Test Driven Development (TDD) para novas features;
- III. Expandir o programa de testes de usabilidade com usuários com deficiência (acessibilidade plena);
- IV. Configurar alertas proativos, baseados em métricas do Sentry, para reduzir ainda mais o MTTR (Mean Time to Recovery).

A Cidade Virtual está em condições de ser utilizada pela comunidade, com alto nível de confiança em sua qualidade e confiabilidade, cumprindo o seu papel como tecnologia social para fortalecimento da economia local.

REFERÊNCIAS

COHN, Mike. **Succeeding with Agile: Software Development Using Scrum**. Addison-Wesley Professional, 2009. (Capítulo sobre pirâmide de testes).

HUMBLE, Jez; FARLEY, David. **Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation**. Addison-Wesley Professional, 2010.

KIM, Gene; HUMBLE, Jez; DeBOIS, Patrick; WILLIS, John. **The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations**. IT Revolution Press, 2016.

REICHHELD, Frederick F. **The One Number You Need to Grow**. Harvard Business Review, dezembro de 2003. (Métrica NPS).

VALENTE, Marco Tulio. **Engenharia de software moderna: princípios e práticas para desenvolvimento de software com produtividade**. v. 1, n. 24, 2020. Capítulo 13 - Testes. Disponível em: <https://link.ufms.br/G5d6h>. Acesso em: 14 mai. 2026.

PLAYWRIGHT DOCUMENTATION. **End-to-end testing for modern web apps**. Disponível em: <https://playwright.dev/>. Acesso em: 14 mai. 2026.

JEST DOCUMENTATION. **Delightful JavaScript Testing**. Disponível em: <https://jestjs.io/>. Acesso em: 14 mai. 2026.

OWASP FOUNDATION. **OWASP ZAP - Zed Attack Proxy**. Disponível em: <https://www.zaproxy.org/>. Acesso em: 14 mai. 2026.

UNIVERSIDADE FEDERAL DE MATO GROSSO DO SUL (UFMS). **Resolução nº 304-COGRAD/UFMS, de 17 de junho de 2021**. Estabelece as Normas para a curricularização da extensão nos Cursos de Graduação. Disponível em: <https://sei.ufms.br>. Acesso em: 14 mai. 2026.